

# Project Report: Temple Visitor Analytics System

Date: April 8, 2026

Team: Harsha, Akshita, Ganesh

## 1. Executive Summary

We have successfully developed and optimized a custom Artificial Intelligence surveillance pipeline to monitor, count, and classify temple visitors. The traditional approach of using "tripwires" (gate-lines) fails in temple environments due to high occlusion, workers sitting in place, and loitering.

To solve this, we architected a hardware-accelerated **Skip-Frame Re-Identification (Re-ID) Pipeline**. This system mathematically fingerprints every individual entering the frame and tracks them dynamically, completely eliminating double-counting and the need for gate-crossings.

## 2. Artificial Intelligence Architecture

Our pipeline relies on three cascaded deep-learning models, optimized specifically for NVIDIA Tensor Cores:

- Detection (YOLOv8s)**: A PyTorch-based spatial detector bounding all humans in the frame. It ignores rigid background motion and is paired with a **Zone Filter** to explicitly blind the AI to designated "worker-only" or "restricted" zones.
- Identity Tracking (OSNet-x1\_0)**: A Convolutional Neural Network that extracts a 512-dimensional vector embedding for every person. Using Hungarian matching and Cosine Distance matrices, it links individuals frame-to-frame.
- Demographic Classification (ConvNeXt-Tiny)**: Compiled to ONNX (Open Neural Network Exchange), this model crops the detected human and predicts gender. To prevent flickering, it strictly requires 5 positive chronological votes before permanently "locking" a visitor's demographic assignment.

## 3. Engineering & Performance Optimizations

Processing high-definition CCTV footage normally requires massive server clusters. By engineering the following constraints into the pipeline, we achieved local hardware processing that outperforms real-time video speeds:

- Hardware Acceleration**: Full utilization of CUDA Cores via PyTorch and `onnxruntime-gpu`

Hardware Acceleration: Full utilization of CPU Cores via Python and **onnxruntime-gpu**.

- **Zero-Copy Skip Frames:** Analyzing video at 25 FPS is redundant for slow-moving crowds. The pipeline mathematically processes every 5th frame (0.2 seconds). Skipped frames bypass CPU memory allocation entirely.
- **Exponential Moving Average (EMA) Banks:** Instead of storing thousands of images of a person to remember them, the system mathematical blends their dynamic features into a single evolving 512-dimension vector.
- **Asynchronous Threaded Queue:** Writing annotated video to a hard drive bottlenecks ML processing. Output frames are pushed to a background thread to be compiled asynchronously, ensuring the GPU never stalls.

## 4. Performance Benchmarks

---

Tested locally on an NVIDIA RTX 3050 Ti Laptop GPU:

| Metric                   | Result                     |
|--------------------------|----------------------------|
| Video Resolution         | 1080p HD                   |
| Pipeline Inference Speed | ~85 FPS                    |
| Real-Time Multiplier     | 3.5x Faster than Real-Time |
| 1- Hour CCTV Video       | 17 Minutes to Process      |
| 24-Hour CCTV Block       | ~6.8 Hours to Process      |

## 5. Output Deliverables

---

The client receives a complete analytical suite:

1. **Cumulative Global Count:** Total unique daily visitors, actively omitting returning staff.
2. **Demographic Breakdown:** Male vs. Female ratios.
3. **High-Traffic Timestamping:** The system monitors velocity spikes in the unique visitor count, automatically flagging specific timestamps as "High Density" events.
4. **Interactive Dashboard:** A local React/Next.js dashboard dynamically visualizes the Server-Sent Events (SSE) data in real-time